AUTOM-01: Automation Landscape

> **Series:** AUTOM — Dynatrace Automation | **Notebook:** 1 of 9 | **Created:** January 2026 | **Last Updated:** 11/13/2026

Dynatrace provides multiple ways to automate configuration management and operational tasks. This series covers all major automation options, helping you choose the right approach for your needs.

Table of Contents

1. Introduction
2. Automation Options Overview
3. Choosing the Right Tool
4. Decision Framework
5. When a Terraform Shop Should Add Monaco
6. First-Time Setup Path — From Zero to GitOps Pipeline
7. Authentication & Token Reference
8. Emerging Capabilities
9. Next Steps

Prerequisites

Before starting this series, ensure you have:

Requirement	Description
Dynatrace SaaS tenant	Active tenant with admin access
Authentication	Access token, platform token, or OAuth client (see Authentication & Token Reference)
Basic familiarity	Understanding of Dynatrace configuration concepts

---

Learning Objectives

By the end of this notebook, you will:

- Understand all available automation options for Dynatrace
- Know when to use each tool or approach
- Be able to choose the right automation strategy for your use case
- Have a framework for evaluating automation approaches

---

1. Introduction

What This Series Covers

Notebook	Focus
AUTOM-01 (this notebook)	Automation landscape overview
AUTOM-02	Settings API - REST API for configuration (incl. the repo's consolidated Settings 2.0 schema catalog)
AUTOM-03	Monaco - Configuration-as-code CLI
AUTOM-04	Terraform Provider - Infrastructure-as-code (incl. the repo's consolidated Terraform resource catalog)
AUTOM-05	Dynatrace Workflows - Event-driven automation
AUTOM-06	Dynatrace SDKs - TypeScript SDK clients; Python via the REST APIs
AUTOM-07	CI/CD Integration - GitOps patterns
AUTOM-08	Migration Automation - Bulk configuration transfer
AUTOM-09	Terraform GitOps Setup Recipe - Repo layout, state backends, lifecycle protections, team onboarding
AUTOM-95 LAB	Terraform IAM Management - Hands-on: OAuth client setup, groups + policies + boundaries + bindings, DSL discovery, bulk export, import
AUTOM-96 LAB	GitHub Actions CI/CD for Dynatrace Terraform - Hands-on: OIDC → Vault → Platform Token, environment-gated apply
AUTOM-97 LAB	Monaco Configuration-as-Code - Hands-on: install, download, validate, deploy, multi-env, GitHub Actions
AUTOM-98 LAB	Terraform for Dynatrace - Hands-on: provider config, mixed-auth, Settings 2.0, IAM, import, state, drift

Why Automate?

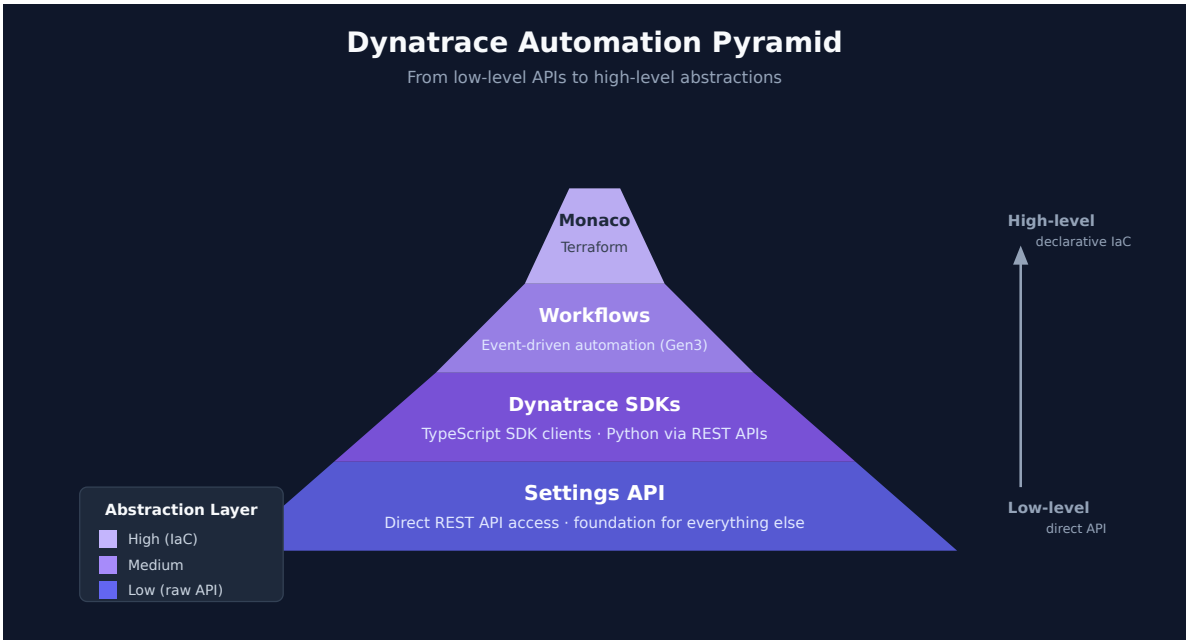
Benefit	Description
Consistency	Same configuration across all environments
Repeatability	Deploy identical setups reliably
Version Control	Track changes in Git, enable rollbacks
Scale	Manage hundreds or thousands of configurations
Speed	Faster deployments, reduced manual effort
Compliance	Auditable changes, approval workflows

---

2. Automation Options Overview

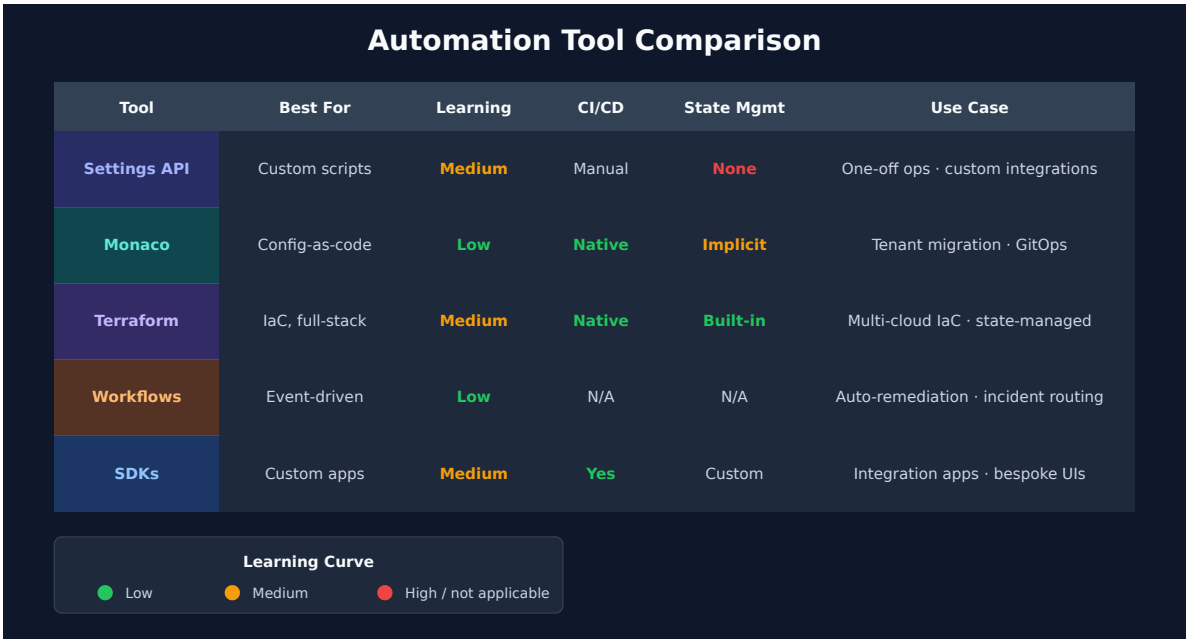
The Automation Pyramid

Dynatrace automation tools form a hierarchy from low-level APIs to high-level abstractions:



Tool Comparison Matrix

Tool	Best For	Learning Curve	CI/CD Ready	Multi-Tenant
Settings API	Custom integrations, scripts	Medium	Manual	Yes
Monaco	Config-as-code, migrations	Low	Yes	Yes
Terraform	IaC environments, full stack	Medium	Yes	Yes
Workflows	Event-driven, auto-remediation	Low	N/A	No
SDKs	Custom apps, complex logic	Medium	Yes	Yes



Settings API

The foundation for all configuration automation. Direct REST API access to Dynatrace settings.

Aspect	Details
Type	REST API
Access	HTTP calls with a platform token or OAuth bearer token (`settings:objects:read` / `settings:objects:write`), or a classic access token (`settings.read` / `settings.write`) where the environment still has them
Schema	JSON with schema validation
Use Case	Custom scripts, one-off operations
Documentation	Settings API Reference (environment API v2, `/api/v2/settings`); the platform API also has a Settings API (`/platform/settings/v1` , platform token or OAuth only) — AUTOM-02 §2 compares them

Monaco

Dynatrace's official configuration-as-code CLI tool. YAML-based configuration management.

Aspect	Details
Type	CLI tool
Configuration	YAML files
Features	Download, deploy (incl. `--dry-run`), delete, generate
Use Case	Config management, migrations
Documentation	Monaco on GitHub

Terraform Provider

Dynatrace's official Terraform provider (`dynatrace-oss/dynatrace` , published as a Partner-tier provider on the Terraform Registry). Infrastructure-as-code approach.

Aspect	Details
Type	Terraform provider
Configuration	HCL (HashiCorp Configuration Language)
Features	Plan, apply, destroy, state management
Use Case	IaC pipelines, environment provisioning
Documentation	Terraform Registry

Dynatrace Workflows

Built-in automation engine for event-driven actions within Dynatrace.

Aspect	Details
Type	Platform feature
Configuration	UI or YAML
Features	Triggers, actions, conditions, JavaScript
Use Case	Auto-remediation, notifications, integrations

Aspect	Details
Documentation	Workflows Documentation

Dynatrace SDKs

Official client libraries for programmatic access to Dynatrace APIs.

Aspect	Details
Type	Client libraries
Languages	TypeScript/JavaScript (`@dynatrace-sdk/*`); Python automation calls the REST APIs directly
Features	Type-safe, auto-generated from OpenAPI
Use Case	Custom applications, complex automation
Documentation	Dynatrace SDK

dtctl

The Dynatrace command-line tool, developed in the open at dynatrace-oss/dtctl. Dynatrace describes it as *"for anybody who wants to use Dynatrace platform functionality via CLI, set up CI/CD pipelines that read or write Dynatrace data, or integrate Dynatrace with AI coding assistants."*

Aspect	Details
Type	CLI tool with kubectl-style verb-noun commands
Access	OAuth browser login, or a platform token where OAuth isn't possible (headless runs); credentials can also come from `DTCTL_ENVIRONMENT` / `DTCTL_TOKEN`
Features	Workflows, DQL queries, dashboards and notebooks, SLOs, Settings objects, buckets, segments and more; `--dry-run` ; JSON/YAML output and an agent output mode (`--agent`)
Use Case	Terminal and scripted operations, CI/CD steps, AI coding assistants
Scope	Latest Dynatrace on SaaS only; Managed has a separate tool, `dtmgd`
Documentation	dtctl CLI

> **Sources:**
>
> - Settings API - POST an object (DT docs) — *"Platform Token / OAuth: Required scope: settings:objects:write"*
> - Upgrade from classic access tokens to platform tokens (DT docs) — (latest environment) *"Platform tokens only. Classic access tokens don't exist, and v2/apiTokens isn't available."*
> - Dynatrace Terraform provider (Dynatrace GitHub) — *"The Dynatrace Terraform provider is officially supported by Dynatrace."*; Partner tier per the Terraform Registry entry, checked 10/02/2026.
> - dtctl CLI (DT docs) — *"dtctl is compatible with Dynatrace SaaS environments only."*, *"In cases where OAuth isn't possible, such as in headless environments, you can also use platform tokens."*

---

3. Choosing the Right Tool

Use Case Mapping

Use Case	Recommended Tool	Why
One-off configuration change	Settings API	No setup overhead — just a curl. Other tools require authoring + state
Repeatable deployments	Either Monaco **or** Terraform	Both are fully capable. Monaco: one manifest → N tenants with no per-env state — pick this when the lower setup cost matters. Terraform: directory-per-env or workspaces achieves the same outcome — pick this when you're already a Terraform shop, since adding Monaco is optional (see §5)
Full environment provisioning	Terraform	Cross-system orchestration — cloud (AWS/Azure/GCP) + Dynatrace + Git in one apply. Monaco is Dynatrace-only
Auto-remediation	Workflows	Built-in event triggers (problem detected → action). Settings API / SDK require building event subscription yourself
Custom application	SDK	Programmatic access — Monaco/Terraform are declarative-config tools, not application frameworks
Tenant migration	Monaco (target = another Dynatrace tenant) **or** `terraform-provider-dynatrace -export` (target = a local Terraform repo)	`monaco download` and the Terraform provider's `-export` both pull every config in one command. `terraform import` (the Terraform CLI feature, distinct from the provider's `-export`) is per-resource and requires authoring matching HCL first
GitOps pipeline	Monaco or Terraform	CI/CD integration
Interactive or scripted operations from a terminal, a pipeline step or an AI coding assistant	dtctl	One CLI across workflows, DQL, dashboards, Settings objects and more, with structured output for scripts and agents. SaaS only

Team Skill Considerations

Team Background	Best Fit
DevOps with Terraform experience	Terraform Provider (Monaco optional — see §5)
Developers comfortable with APIs	Settings API or SDK
SRE teams wanting GitOps	Monaco
Operations needing quick automation	Workflows
Mixed skill levels	Monaco (lowest barrier)

Feature Coverage Comparison

Not all tools support all Dynatrace features equally:

Configuration Type	Settings API	Monaco	Terraform
Settings 2.0 objects	Full	Full	Full
Classic config (legacy)	N/A	Full	Full

Configuration Type	Settings API	Monaco	Terraform
Dashboards	N/A — Document API (classic dashboards: Configuration API)	Full (`document` type)	Full
Synthetic monitors	Full	Full	Full
Alerting profiles [1]	Full	Full	Full
Management zones [1]	Full	Full	Full
Auto-tagging rules [1]	Full	Full	Full
SLOs (classic `builtin:monitoring.slo`) [1]	Full	Full	Full
Workflows	N/A — Automation API	Full (`automation` type)	Full (`dynatrace_automation_workflow`)
OpenPipeline	Full	Full	Full

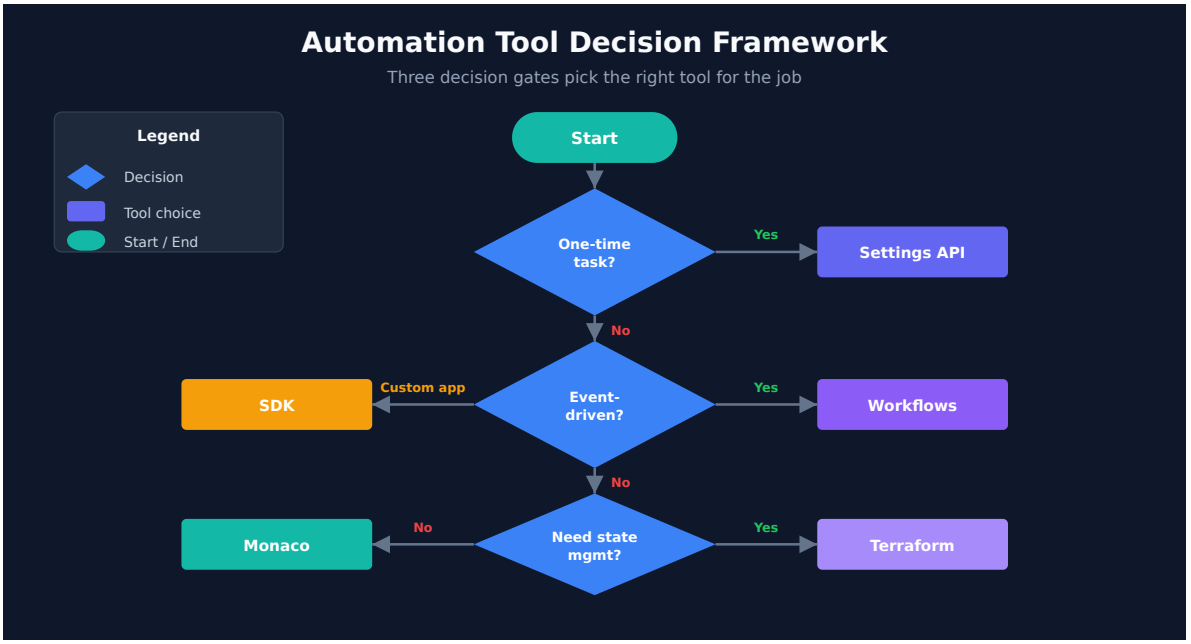
[1] The schema behind this row is removed in Latest Dynatrace — it works until your environment upgrades. See the Settings 2.0 schema catalog in **AUTOM-02** for the status of each schema and its successor.

> **Note:** Monaco and Terraform call the Settings API for Settings 2.0 objects and the platform APIs (Automation, Document, Grail storage) for platform resources — which is why Monaco takes a platform token or OAuth client alongside its access token. Feature parity for Settings objects depends on schema availability.
>
> **Sources:**
>
> - Monaco YAML configuration (DT docs) — *"Automation defines a workflow."*; *"Document defines a dashboard, a notebook and a launchpad."*
> - Monaco manage resources (DT docs) — *"a platform token or OAuth client for the Dynatrace Platform APIs, such as automation workflows, documents, and Grail buckets."*
> - `dynatrace_automation_workflow` (Dynatrace GitHub)
> - Settings 2.0 schemas that are removed in Latest Dynatrace (DT docs) — *"None of the schemas on this page are visible in Latest Dynatrace."*

---

4. Decision Framework

Use this flowchart to choose the right automation approach:



Combining Tools

These tools aren't mutually exclusive. Common combinations:

Combination	Use Case
Monaco + Workflows	Deploy configs with auto-remediation
Terraform + Monaco	Infra provisioning + config management
SDK + Settings API	Custom app with direct API fallback
Monaco + CI/CD	GitOps config deployments

Anti-Patterns to Avoid

Anti-Pattern	Problem	Solution
Mixing Monaco and Terraform for same config	State conflicts	Choose one for each config type
Manual changes alongside automation	Configuration drift	Use automation exclusively
No version control	No rollback capability	Always use Git
Hardcoded secrets	Security risk	Use environment variables or vaults

5. When a Terraform Shop Should Add Monaco

Monaco is optional for Terraform shops. A confident Terraform shop can cover the full Dynatrace surface — provisioning, configuration, drift detection, multi-env promotion — without ever adopting Monaco. The Terraform provider's built-in `-export` utility (see §3 "Tenant migration" row and the bulk-download workflow below) removes the historical reason teams reached for Monaco as a Terraform on-ramp.

This section names **four specific patterns** where *adding* Monaco alongside an established Terraform footprint can pay off. **None are blockers.** If none of these patterns describe your situation, skip this section and stay Terraform-only — that's a

complete, supported path.

The general guidance in §4 (Combining Tools) still applies: pick one tool per config and avoid overlap. The patterns below name the seams where Monaco genuinely earns its place — not a recommendation to adopt Monaco by default.

Four Patterns Where Monaco Wins for a Terraform Shop

Pattern	Why Monaco wins
Bulk download from an existing tenant	When the *target* is Terraform, the Dynatrace Terraform provider's built-in export utility (`terraform-provider-dynatrace -export`) pulls every supported resource directly into ready-to-use HCL — no Monaco intermediate, no `terraform import` resource-by-resource authoring. When the *target* is Monaco, `monaco download` is the equivalent. For Terraform shops onboarding an inherited tenant, the provider's `-export` is the on-ramp — Monaco-as-intermediate adds a manual HCL-authoring step that the provider already does for you.
N tenants, identical configs	Monaco's `manifest.yaml` lists multiple environments and deploys to all with variable substitution — no per-environment state, no workspaces. For *"we have prod + EU-prod + APAC-prod and they should all look the same"*, Monaco is less ceremony. Terraform workspaces can do it, but each has its own state and the divergence-over-time tax is real.
App-team self-service	Letting product teams commit Dynatrace configs alongside their app code with a CI job running `monaco deploy` on merge — no state backend per team, no state-locking infrastructure. The platform team keeps Terraform-managed shared infra; app teams get a low-floor path for their own SLOs / dashboards / management zones.
Bleeding-edge Settings 2.0 schemas	In community practice, Monaco's schema-id configs reach a new schema first, while the Terraform provider's typed resources can lag (its `dynatrace_generic_setting` resource is documented for schemas contributed by custom platform apps, so coverage of new `builtin:` schemas is not documented either way). For a config that has no typed resource yet, Monaco is a fallback — verify against the provider's current resource list first.

What's NOT a Good Reason for a Terraform Shop

Reason	Why it's weak
"Monaco's YAML is nicer than HCL"	Preference, not technical. For an existing shop, stick with what works.
"Monaco doesn't need state"	True, but you'll want some equivalent (Git history at minimum) for audit. The state-less property is a benefit only when state management is genuinely a barrier (the self-service pattern above).
"Adding Monaco gives us redundancy / a backup"	Two tools managing overlapping resources is the documented anti-pattern in §4 Combining Tools. The coordination cost dwarfs the perceived resilience.
"We need Monaco to bootstrap a Terraform repo from an existing tenant"	No — the Terraform provider's own `-export` does this directly. See the bulk-download workflow below.

The Practical Boundary

For an experienced Terraform shop adopting Monaco selectively, the rule is **boundary, not choice** — each config belongs to exactly one tool, and the line runs along these natural seams:

Use Monaco for	Use Terraform for
Tenant cloning / bulk migration	Cross-cloud orchestration (AWS / Bitbucket / Dynatrace in one apply — see AUTOM-07 §5 *Combined Workspace*)
Per-app self-service configs	Platform-team shared infra
Bleeding-edge schemas	Anything with cross-system dependencies
Multi-tenant N×deploy of identical sets	State-backed, dependency-graphed infra

Use Monaco for	Use Terraform for
—	Configs you actively manage, including drift detection (`terraform plan`; Monaco has no drift view — `--dry-run` does not contact the tenant)

Document the boundary explicitly (in `DECISIONS.md` or your tenant runbook) so teams know which tool owns which config category — that's what prevents the anti-pattern of overlap.

When You're Doing the Bulk Download

Two paths, picked by target:

Target = Terraform repo (most common for a Terraform shop):

1. `terraform-provider-dynatrace -export` — pulls every supported resource directly into HCL. Output lands in `DYNATRACE_TARGET_FOLDER` (defaults to `.configuration/`). Use `-flat` for a single directory; default is a module structure. Use `-list-exclusions` to see what's excluded by default (notably dashboards — opt in explicitly when you want them). Add `-admin-access` (requires the `settings:objects:admin` permission) so OpenPipeline resources owned by other users are exported too — without it only the current user's are.
2. Review and clean up — the export produces `.flawed/` for deprecated configs and `.requires_attention/` (the docs page spells it `.required_attention`; AUTOM-04 § 8 quotes the utility source) for items missing essentials (e.g., credential payloads the API can't return). Triage both.
3. Commit to Git as your *source-of-truth* baseline.
4. From here you're on the standard Terraform workflow — `terraform plan` shows drift vs the tenant; `terraform apply` reconciles. See AUTOM-04 + AUTOM-09 for the operational layer.

Target = decision-not-yet-made (the "see what's there first" pattern):

1. `monaco download --manifest manifest.yaml --environment <tenant>` — pulls every config to YAML. Human-readable; faster to skim than HCL.
2. Review the YAML to understand the tenant's footprint.
3. Decide:
   - **Stay on Monaco** if the configs are stable and the deploy cadence is low → Monaco's lower-ceremony loop wins.
   - **Switch to the Terraform provider's `-export`** if you're committing to Terraform → use the YAML as a checklist for what to expect in the HCL output.

Don't use Monaco-download-as-Terraform-bootstrap. Monaco's YAML and the Terraform provider's HCL don't share a converter (no Dynatrace-supplied tool, and no community OSS bridge found in GitHub searches as of September 2026). Hand-authoring HCL from Monaco YAML loses to running `-export` directly.

> **Sources:**
>
> - Dynatrace Terraform provider export utility (DT docs) — *"./terraform-provider-dynatrace -export -ref -exclude dynatrace_platform_slo dynatrace_automation_workflow"*; *"-admin-access : Export OpenPipeline resources of all owners, not just the current user's. Requires the settings:objects:admin permission."*; module-structure vs `-flat`; `.flawed` / `.requires_attention` triage dirs; default exclusions.
> - `dynatrace_generic_setting` (Dynatrace GitHub) — *"The resource `dynatrace_generic_setting` covers configuration for Schemas contributed by Custom Platform Apps"*
> - Monaco repo README (Dynatrace GitHub) — confirms Monaco and Terraform provider are separate tools; no built-in conversion. **Observed 09/28/2026:** GitHub repository searches for `monaco to terraform`, `monaco terraform dynatrace`, `dynatrace monaco convert` and `dynatrace monaco migration` returned no Monaco-to-Terraform converter (first checked 05/12/2026). An absence in search results is an observation, not a guarantee — re-check before relying on it.

---

6. First-Time Setup Path — From Zero to GitOps Pipeline

Once you've picked Monaco or Terraform (per §3-§5), here's the sequenced path from "no automation today" to "PR-driven plan-then-apply pipeline." Both paths assume the tenant already has manually-created configuration that you want to bring into source control.

Path A — Terraform target (most common for shops with existing IaC)

> **Monaco does not appear in this path.** A Terraform-only setup is complete and supported end-to-end. If you later hit one of the patterns in §5, you can selectively add Monaco alongside — but the loop below stands on its own.

1. **Install the Terraform CLI** — see **AUTOM-04 §2 Getting Started** for OS-specific install commands.
2. **Configure the Dynatrace provider** — see **AUTOM-04 §3 Provider Configuration**. Set `DYNATRACE_ENV_URL` + a Platform Token (`dt0s16`) for platform resources (workflows, documents, segments, buckets, OpenPipeline). Add a classic API Token (`dt0c01`) for classic resources — the provider documents that platform tokens cannot be used for them — and for the resources its v1.88.0 release moved to API-token-only (16 resources incl. synthetic and network monitors, SLOs, credentials and network zones; full list in **AUTOM-04 §3**). IAM resources need an OAuth client. Classic API tokens no longer exist once the environment is on Latest Dynatrace.
3. **Bulk-export the tenant's existing config to HCL** — run `terraform-provider-dynatrace -export` (the provider's built-in export utility — **NOT** `terraform import` , which is per-resource). Output lands in `.configuration/` ; triage `.flawed/` (deprecated configs) and `.requires_attention/` (sensitive fields the API can't return) before committing. See **AUTOM-04 §8 Next Steps** for invocation details and **AUTOM-01 §5 bulk-download workflow** for the full triage flow.
4. **Stand up the repo layout** — choose single-repo (`modules/` + `envs/`) or two-repo (modules separate from consumer). See **AUTOM-09 §2 Opinionated Repo Layout**.
5. **Configure the state backend** — S3+DynamoDB, GCS, Azure Storage, or HCP Terraform. See **AUTOM-09 §3 State Backend Setup** for `backend.tf` examples per backend.
6. **Lock down version constraints** — `required_providers` + `~>` constraint discipline. See **AUTOM-09 §4 Provider Configuration and Version Discipline**.
7. **Modularize repeated patterns** — start with one module per resource family (management-zone, alerting-profile, slo). See **AUTOM-09 §5 Module Strategy**.
8. **Establish multi-environment promotion** — directory-per-env with shared modules + per-env tfvars. See **AUTOM-09 §6 Multi-Environment Promotion**.
9. **Wire up CI/CD** — pick your platform and follow its recipe: **AUTOM-07 §3 GitHub Actions** · **§4 GitLab CI/CD** · **§5 Bitbucket Pipelines** · **§6 Atlassian Bamboo** · **§7 Azure DevOps Pipelines**. Each section covers plan-on-PR + manual-gated apply + plan-artifact reuse.
10. **Add lifecycle protections + secrets discipline + onboarding** before the first real prod apply. See **AUTOM-09 §8 Secrets Handling** · **§9 Lifecycle Protections** · **§10 Onboarding New App Teams** · **§12 Operational Realities** (stuck state lock, break-glass, DR).

Path B — Monaco target (config-only, single-tool simplicity)

1. **Install Monaco** — see **AUTOM-03 §2 Getting Started**. `curl` the release binary (macOS/Linux) or download the `.exe` (Windows); there is no Homebrew formula.
2. **Set environment variables** — tenant URL + access token, plus a platform token or OAuth client for platform configs (workflows, documents, buckets). See **AUTOM-03 §2 Environment Setup**.
3. **Bulk-download the tenant's existing config to YAML** — `monaco download --manifest manifest.yaml --environment <tenant>` . See **AUTOM-08 Migration Automation** for the full migration flow.
4. **Review and clean up** — Monaco's YAML is human-readable; delete noise, parameterize sensitive values, split into logical projects. See **AUTOM-03 §3 Project Structure**.
5. **Author your** **`manifest.yaml`** — list every environment Monaco should deploy to. Per-env values via `parameters` block. See **AUTOM-03 §4 Configuration Files**.

6. **Commit to Git as your source-of-truth baseline.**
7. **Wire up CI/CD** — pick your platform and use the **Monaco-deploy pattern** from: **AUTOM-07 §3 GitHub Actions** · **§4 GitLab CI/CD** · **§5.1 Bitbucket Pipelines — Monaco Deploy** · **§6 Atlassian Bamboo** (adapt the Plan Specs YAML to call `monaco deploy` instead of `terraform plan/apply`).
8. **Add review + approval discipline** — Monaco doesn't have state-file locking, but pipeline-level approval gates work the same way. PR triggers `monaco deploy --dry-run` (Monaco has no `validate` command); merge to main triggers `monaco deploy` (gated by manual approval for production).
9. **Plan for tool evolution** — if your needs grow beyond Dynatrace-config-only, **AUTOM-01 §5** documents the patterns where adding Terraform alongside Monaco makes sense.

Common pitfalls (both paths)

Pitfall	Why it bites	Fix
Pushing a token in `terraform.tfvars` to Git	Source-control leak of long-lived credentials	Use `*.tfvars` in `.gitignore`; pass tokens via env vars or CI/CD secret store
First `apply` without `prevent_destroy` on critical resources	One bad PR can delete production management zones / IAM policies	Add `lifecycle { prevent_destroy = true }` on prod resources before the first prod apply. See **AUTOM-09 §9**
Skipping the `.requires_attention/` triage after `-export`	Apply fails on missing credential payloads	Open the `.requires_attention/` directory; fill in stub values; re-run validate; only then commit
One state file across all environments	dev apply can corrupt prod state	Per-env state file from day 1. See **AUTOM-09 §3** + **§6**
Apply runs without artifact reuse from the reviewed plan	The plan reviewers saw isn't what gets applied	Use the plan-artifact pattern: plan publishes `tfplan` → apply subscribes to it. See **AUTOM-07 §3-§7** for platform-specific syntax

> **Sources:**
> - Dynatrace Terraform provider (Dynatrace GitHub) — *"Platform tokens can't be used for IAM (Account Management) or classic resources."*
> - Terraform provider v1.88.0 release notes (Dynatrace GitHub) — *"For these resources, an API token must be provided"*
> - Monaco manage resources (DT docs) — *"Access tokens and platform tokens are not interchangeable."*
> - Upgrade from classic access tokens to platform tokens (DT docs) — (latest environment) *"Platform tokens only. Classic access tokens don't exist, and v2/apiTokens isn't available."*

---

7. Authentication & Token Reference

Dynatrace supports three types of credentials for automation tools. Which you need depends on the tool and the resources you manage.

Token Types

Token Type	Description	Use Case
Access Token (Classic)	Scope-based token with explicit permissions (e.g., `settings.read`). Exists only in classic and hybrid environments — not once the environment is on Latest Dynatrace	Settings API, Monaco, Terraform (settings/classic)

Token Type	Description	Use Case
Platform Token	Long-lived token bound to a user's permissions; simpler to create	Same as access token, but scopes are limited to user's existing permissions
OAuth Client	Client ID + secret exchanged for short-lived tokens	**Required** for account IAM resources; workflows, documents and segments accept a Platform Token or an OAuth client

Tool Authentication Matrix

Tool	Access/Platform Token	OAuth Client	Notes
Settings API	Platform token: `settings:objects:read`, `settings:objects:write` (`Authorization: Bearer`); classic access token: `settings.read`, `settings.write` (`Authorization: Api-Token`)	`settings:objects:read`, `settings:objects:write`	Direct REST calls; see AUTOM-02
Monaco	Access token: `DataExport`, `ReadConfig`, `WriteConfig`, `settings.read`, `settings.write` (`auth.token`); platform token for platform configs (`auth.platformToken`)	Alternative to the platform token for platform configs (`auth.oAuth`)	Workflows, documents, Grail buckets and segments need `platformToken` or `oAuth`; access and platform tokens are not interchangeable
Terraform (settings/classic)	`settings.read`, `settings.write`, `ReadConfig`, `WriteConfig`	N/A	IaC for config objects
Terraform (automation/documents)	Platform Token with `DYNATRACE_HTTP_OAUTH_PREFERENCE=true` (see AUTOM-04 §3)	`automation:workflows:read/write`, `document:documents:read/write`	Platform Token **or** OAuth
Terraform (account management)	N/A	IAM scopes + `DT_ACCOUNT_ID`	OAuth **required**
Workflows	Built-in (no external token needed)	N/A	Platform feature
SDKs	Depends on operations performed	N/A	Client libraries

Token Best Practices

Practice	Description
Least privilege	Only grant required scopes
Environment-specific	Separate tokens per environment
Rotation	Rotate tokens regularly
Secret storage	Use HashiCorp Vault, AWS Secrets Manager, etc.
Platform tokens	Prefer over classic access tokens for simpler management
OAuth for service identities	Prefer OAuth clients for CI/CD service identities managing Workflows or Documents; a Platform Token also works

> **Key Distinction:** Both platform tokens and OAuth clients act as a user. A platform token is limited to the permissions of the user or service user it belongs to (a scope only grants access if that user already has the permission). A client-credentials OAuth client is bound to a subject user, and its tokens get the intersection of the client's scopes and that user's permissions. For CI/CD, bind either one to a service user.
>
> **Sources:**
>
> - Settings API - POST an object (DT docs) — *"Platform Token / OAuth: Required scope: settings:objects:write"*
> - Upgrade from classic access tokens to platform tokens (DT docs) — *"platform tokens are available alongside classic access tokens"* (hybrid); (latest environment) *"Platform tokens only. Classic access tokens don't exist, and v2/apiTokens isn't available."*
> - Monaco manage resources (DT docs) — access-token minimum includes *"Access problem and event feed, metrics, and topology (DataExport)"*
> - OAuth clients (DT docs) — *"For Client credentials only: provide the Subject user email"*
> - Dynatrace MCP server (DT docs) — *"the effective token permissions consist of the intersection of the permissions configured on the OAuth client and the permissions of the user who created the client."*

---

8. Emerging Capabilities

Dynatrace Intelligence Agents

Announced at Perform 2026, **Dynatrace Intelligence** is an agentic operations system that fuses deterministic and agentic AI. Intelligence Agents transform insights into autonomous outcomes across IT and business operations.

Aspect	Details
Type	Agentic AI layer built into the platform
Capabilities	Autonomous SRE, security, and development agents
Integration	Works through Dynatrace Workflows for agentic workflows
Governance	Built-in guardrails for supervised or autonomous operation
Use Case	Auto-prevention, auto-remediation, auto-optimization

> **Note:** Dynatrace Intelligence Agents represent a shift from rule-based automation (Workflows, Monaco, Terraform) to AI-driven autonomous operations. They complement existing tools rather than replacing them.

Dynatrace MCP Server

The Dynatrace MCP Server implements the Model Context Protocol (MCP), enabling AI assistants (Claude, GitHub Copilot, Amazon Q) to interact with Dynatrace using natural language.

Aspect	Details
Type	Dynatrace-hosted MCP server, reached through your environment's MCP gateway URL with a platform token or OAuth bearer token (the earlier open-source local server, `@dynatrace-oss/dynatrace-mcp-server`, is deprecated)
Access	AI assistants can query problems, metrics, traces, logs, topology
Use Case	AI-assisted development, triage, incident management
Documentation	MCP Server Documentation

How Emerging Capabilities Fit

Automation Maturity	Approach
Manual	Settings API, one-off scripts
Automated	Monaco, Terraform, CI/CD pipelines
Supervised	Workflows with human approval gates
Autonomous	Intelligence Agents with guardrails
AI-Assisted	MCP Server for developer and SRE tooling

> **Sources:**
> - Dynatrace MCP server (DT docs) — *"Every request to the MCP server needs a bearer token in the authorization header."*
> - dynatrace-mcp README (Dynatrace GitHub) — *"This repository is deprecated. Version 2.1.2 was the final release — no further updates will be made."*

---

9. Next Steps

Learning Path by Goal

Your Goal	Recommended Path
Quick script automation	AUTOM-02 (Settings API)
GitOps config management	AUTOM-03 (Monaco) → AUTOM-07 (CI/CD)
Full IaC environment	AUTOM-04 (Terraform) → AUTOM-07 (CI/CD)
Auto-remediation	AUTOM-05 (Workflows)
Custom application	AUTOM-06 (SDKs)
Tenant migration	AUTOM-08 (Migration)

Continue the Series

Next Notebook	Focus
AUTOM-02: Settings API	Deep dive into REST API configuration

Additional Resources

- Dynatrace API Documentation
- Monaco GitHub Repository
- Terraform Provider Documentation
- Dynatrace Developer Portal
- dtctl CLI

Summary

In this notebook, you learned:

- The automation options available for Dynatrace configuration
- When to use Settings API, Monaco, Terraform, Workflows, or SDKs
- How to choose the right tool based on your use case and team skills
- Best practices for combining automation tools

> **Key Takeaway:** Choose the automation tool that matches your team's skills and your operational model. Terraform shops can stay Terraform-only (§5); in community practice, teams without IaC experience often find Monaco the lower-ceremony place to start.

---

*Continue to **AUTOM-02: Settings API** to learn direct REST API configuration.*

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace.

Always verify information against official Dynatrace documentation.